SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Vishwesh/192511383 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
VISHWESH.K

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

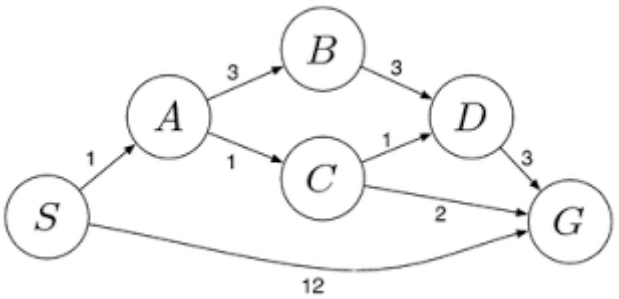
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

SIMATS ENGINEERING

Assessment Tool 1 — Course: Artificial Intelligence (CSA17) | CO1 Solutions

Course Outcome	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)	**Weightage & Marks**	35% | Total Marks: 50

Question 1: Water Jug Problem

Problem Statement: You are given 2 jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug?

State Representation: (x, y) where x is the volume of water in the 4-gallon jug ($0 \le x \le 4$) and y is the volume of water in the 3-gallon jug ($0 \le y \le 3$).

Initial State: $(0, 0)$ | **Goal State:** $(2, y)$ where y can be anything (typically $(2, 0)$).

Step-by-Step Solution Sequence:

1. **(0, 0) → (0, 3):** Fill the 3-gallon jug from the pump.
2. **(0, 3) → (3, 0):** Pour all water from the 3-gallon jug into the 4-gallon jug.
3. **(3, 0) → (3, 3):** Fill the 3-gallon jug completely from the pump.
4. **(3, 3) → (4, 2):** Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. (Since the 4-gallon jug already has 3 gallons, it takes 1 gallon from the 3-gallon jug, leaving exactly 2 gallons in the 3-gallon jug).
5. **(4, 2) → (0, 2):** Empty the 4-gallon jug completely onto the ground.
6. **(0, 2) → (2, 0):** Pour the 2 gallons of water from the 3-gallon jug into the 4-gallon jug.

Result: Exactly 2 gallons of water are now present in the 4-gallon jug $(2, 0)$.

Question 2: Mars Rover Analysis (PEAS Framework & Architecture)

i. What are the percepts for this agent?

The percepts include high-resolution camera images of the Martian terrain and rocks, spectrometer readings for chemical/mineral composition, atmospheric pressure and temperature sensor data, wheel slip and touch sensors (tactile feedback), and GPS/inertial navigation data.

ii. Characterize the operating environment:

- **Fully Observable vs. Partially Observable:** Partially Observable (the rover can only sense its immediate surroundings and localized subsurface data).
- **Deterministic vs. Stochastic:** Stochastic (terrain conditions, rock stability, and weather/wind are unpredictable).
- **Episodic vs. Sequential:** Sequential (current actions like drilling or movement significantly impact future exploration paths and battery life).
- **Static vs. Dynamic:** Dynamic (dust storms, shifting sand dunes, and temperature changes occur while the rover is operating).
- **Discrete vs. Continuous:** Continuous (time, steering angles, power levels, and sensor values are continuous).
- **Single-agent vs. Multi-agent:** Single-agent (though orbiters or landers may exist, the rover acts autonomously in its immediate task environment).

iii. What are the actions the agent can take?

Drive/navigate (move forward, turn, reverse), adjust camera angles/zoom, deploy robotic arm, use drill/abrasion tool to expose fresh rock surfaces, fire spectrometer lasers (e.g., ChemCam), store soil/rock samples in sample tubes, and transmit data back to Earth.

iv. How can one evaluate the performance of the agent?

Performance metrics include: volume and diversity of high-quality scientific samples collected, safe distance traveled without damage or getting stuck, efficiency of power consumption versus solar recharge, accuracy of geological identification, and amount of useful scientific data transmitted back to Earth.

v. What sort of agent architecture is most suitable? Why?

Model-based Reflex Agent with State (and Goal-based elements): This is most suitable because the Mars environment is partially observable and dynamic. The rover needs an internal state to track where it has traveled, battery levels, and unvisited targets (goals), while using models of how its actions affect the environment (e.g., wheel traction on loose soil).

Question 3: 8-Queens Problem Formulation and Search Strategy

Problem Components (Formal Formulation):

- **Initial State:** An empty chessboard with 0 queens placed, or a board configuration with 8 queens placed one per column.
- **Actions:** Add a queen to any square in an empty column, or move a queen within its column. In the incremental formulation: add a queen to the next column in a row/position that is not attacked.
- **Transition Model:** Returns the board configuration after placing a queen at a specific row and column.
- **Goal Test:** Check if all 8 queens are placed on the board such that no two queens attack each other (no two queens share the same row, column, or diagonal).
- **Path Cost:** Each step of placing a queen has a cost of 1, or cost is evaluated solely upon reaching the complete 8-queen goal configuration.

Search Strategy Exploration:

Depth-First Search (DFS) with backtracking or Backtracking Search (Constraint Satisfaction Problem approach) is highly effective. Instead of exploring all $64_{C_8} = 4{,}426{,}1653$ possible states, backtracking places queens column by column, pruning branches as soon as a conflict (same row or diagonal) is detected, reducing the search space to only 2,057 candidate states.

Question 4: OLA Cab Booking Problem-Solving Agent & Pseudocode

Type of Problem-Solving Agent:

Goal-Based Agent (specifically combined with a Utility-Based optimization layer). The agent takes user pickup/drop locations and preferred cab type (mini, micro, sedan, shared, prime) as goals, searches for available drivers matching spatial and category constraints, and optimizes routes and pricing based on utility factors (ETA, fare cost, vehicle rating).

Pseudocode:

```
Function OLA_Cab_Booking_Agent(PickupLocation, DropLocation, PreferredCabType):
    // Step 1: Initialize Goal and Constraints
    Goal = Route(PickupLocation, DropLocation)
    Constraints = {CabType: PreferredCabType, MaxRadius: 5km}

    // Step 2: Search for available drivers matching criteria
    NearbyDrivers = SpatialDatabase.FindDrivers(PickupLocation,
Constraints.MaxRadius)
    ValidDrivers = []

    For each Driver in NearbyDrivers:
        If Driver.VehicleType == PreferredCabType AND Driver.Status ==
"AVAILABLE":
            ValidDrivers.append(Driver)

    If ValidDrivers is Empty:
        Return "No cabs available for selected category. Try another cab type."

    // Step 3: Evaluate Utility for each valid driver (cost vs ETA)
    BestDriver = None
    MaxUtility = -INFINITY

    For each Driver in ValidDrivers:
        ETA = CalculateETA(Driver.Location, PickupLocation)
        Fare = CalculateFare(PickupLocation, DropLocation, PreferredCabType)
        Utility = EvaluateUtility(ETA, Fare, Driver.Rating)

        If Utility > MaxUtility:
            MaxUtility = Utility
            BestDriver = Driver

    // Step 4: Dispatch Booking Request
    SendBookingRequest(BestDriver, PickupLocation, DropLocation)
    Return BookingConfirmation(BestDriver, EstimatedArrivalTime)
End Function
```

Question 5: Uniform Cost Search (UCS) for Logistics Network

Problem Statement: Find the least-cost path from starting warehouse S to destination warehouse G using Uniform Cost Search (UCS) on the given weighted graph.

Graph Edges & Costs:

Edge	Cost	Edge	Cost
S → A	1	C → D	1
S → G	12	C → G	2
A → B	3	D → G	3
A → C	1	B → D	3

UCS Step-by-Step Execution (Priority Queue ordered by cumulative path cost $g(n)$):

- **Initialization:** Priority Queue (PQ) = $[(S, cost = 0)]$. Explored Set = $\{\}$.
- **Iteration 1:** Pop $(S, 0)$. Expand S.
  Neighbors: $A\ (cost = 1)$, $G\ (cost = 12)$.
  $PQ = [(A, 1), (G, 12)]$. Explored = $\{S\}$.
- **Iteration 2:** Pop $(A, 1)$. Expand A.
  Neighbors: $B\ (cost = 1 + 3 = 4)$, $C\ (cost = 1 + 1 = 2)$.
  $PQ = [(C, 2), (B, 4), (G, 12)]$. Explored = $\{S, A\}$.
- **Iteration 3:** Pop $(C, 2)$. Expand C.
  Neighbors: $D\ (cost = 2 + 1 = 3)$, $G\ (cost = 2 + 2 = 4)$.
  $PQ = [(D, 3), (B, 4), (G, 4\ [via\ C]), (G, 12)]$. Explored = $\{S, A, C\}$.
- **Iteration 4:** Pop $(D, 3)$. Expand D.
  Neighbors: $G\ (cost = 3 + 3 = 6)$. (Since path to G with cost 4 is already queued/found, ignore or keep better).
  $PQ = [(B, 4), (G, 4\ [via\ C]), (G, 6\ [via\ D]), (G, 12)]$. Explored = $\{S, A, C, D\}$.
- **Iteration 5:** Pop $(B, 4)$. Expand B.
  Neighbors: $D\ (cost = 4 + 3 = 7)$ (already explored/worse).
  $PQ = [(G, 4\ [via\ C]), (G, 6\ [via\ D]), (G, 12)]$. Explored = $\{S, A, C, D, B\}$.
- **Iteration 6:** Pop $(G, 4)$. Since G is the goal node, we halt and return the path.

Final Result:

- **Optimal Path:** $S \rightarrow A \rightarrow C \rightarrow G$
- **Total Minimum Cost:** $1 + 1 + 2 = 4$